

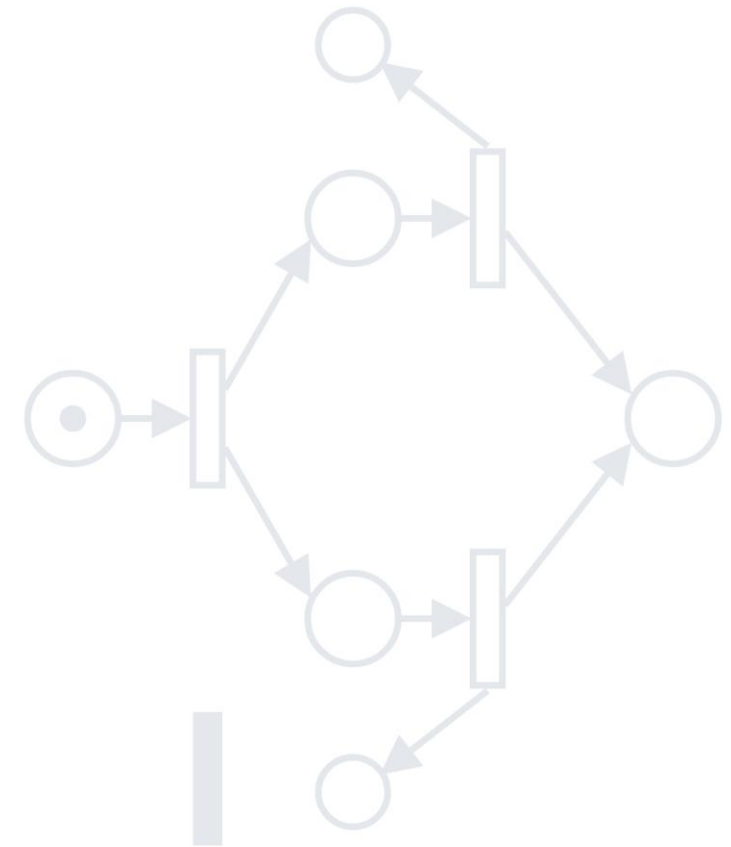
Master Thesis · Master in Data Science

Neuro-Symbolic Predictive Process Monitoring with Procedural Background Knowledge

Matteo Zorzi

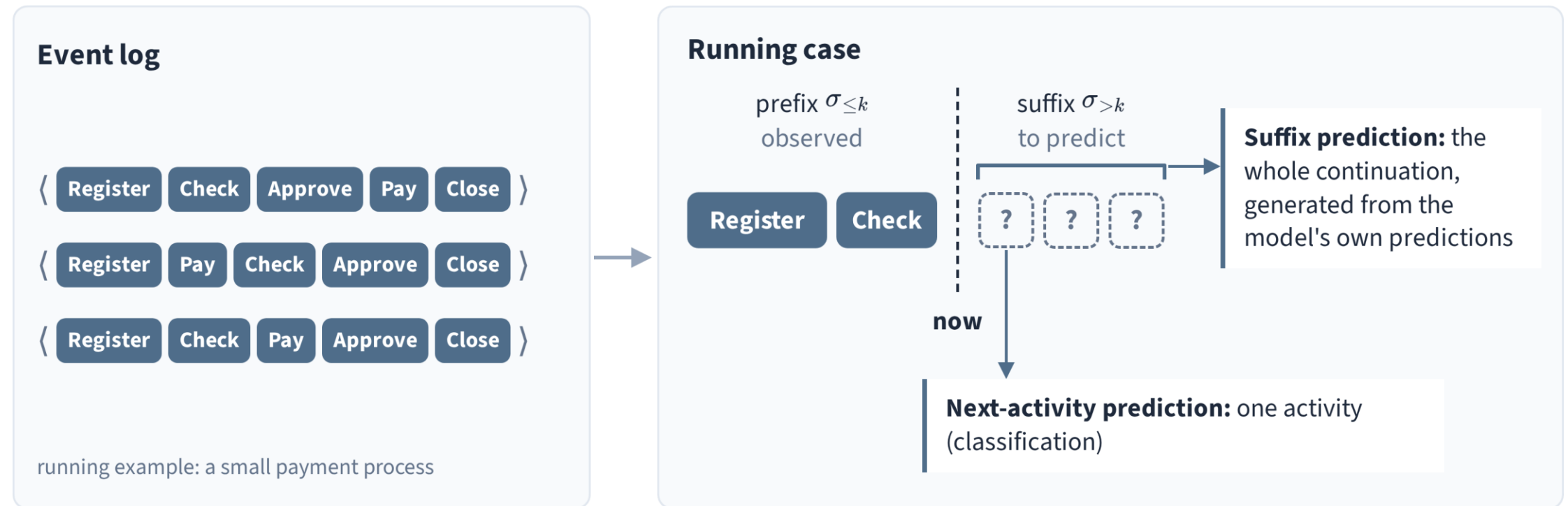
Supervisor: Ivan Donadello, Ph.D. · Co-supervisor: Prof. Fabrizio Maria Maggi, Ph.D.

Faculty of Engineering · Free University of Bozen-Bolzano · 7 October 2026

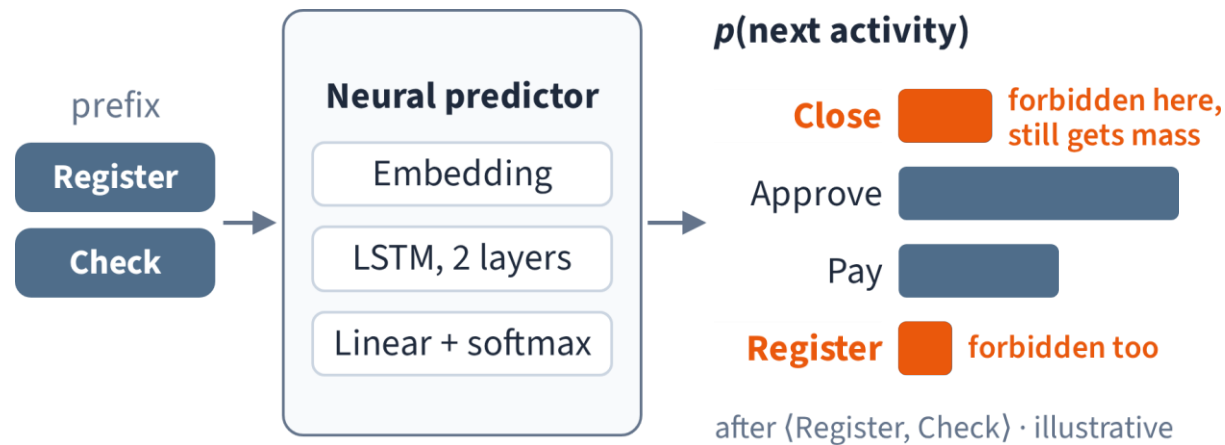


Predictive process monitoring forecasts how a running case will continue

Event log = recorded executions (cases) of a business process · each case is a trace of activities



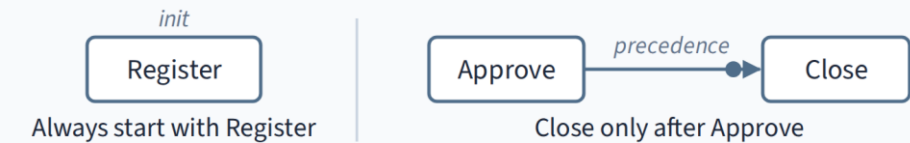
Deep learning methods ignore process models



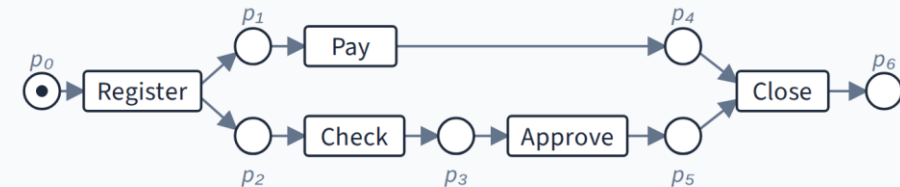
Standard predictor: learns from activity labels only (Tax et al., 2017)

Background knowledge comes in two forms

Declarative: rules a trace must satisfy (e.g., Declare)



Procedural: orders a trace may follow (e.g., Petri net)



- Neuro-symbolic PPM gives the network this knowledge
- Current NeSy methods use only the declarative form, neglecting the procedural one

Gaps of the state of the art

G1 • Only declarative knowledge

The logic losses have only seen declarative knowledge (Declare $\rightarrow$ LTLf $\rightarrow$ automaton), never a Petri net.

G2 • Limited noise conditions

Noise $\leq 40\%$; every trace violating the mined specification removed $\rightarrow$ the baseline starts near full compliance.

G3 • Limited evaluation

Feature based and loss based methods share no log, baseline, architecture or metric $\rightarrow$ no gain is attributable to the channel; none reports computational time.

Three research questions

RQ1 · Channel

Does the same knowledge, injected as an input feature, achieve what it achieves as a loss?

RQ2 · Form

Among the losses, does the form of the information matter: projected onto activity pairs, or indexed by the state the prefix reaches?

RQ3 · Cost

What are the computational times of NeSy methods with procedural knowledge?

Contribution: procedural knowledge in a controlled benchmark

C1 · Procedural knowledge

The state of the art uses Declare models turned into automata, but the same can be done procedurally with Petri nets: this thesis is the first to do it.

closes G1

C2 · Stress test

Noise up to 80%, no trace filtering, knowledge mined under two protocols.

closes G2

C3 · Controlled evaluation

7 knowledge models + baseline: same net, same recurrent trunk, same protocol, only the channel varies; computational time measured for every variant.

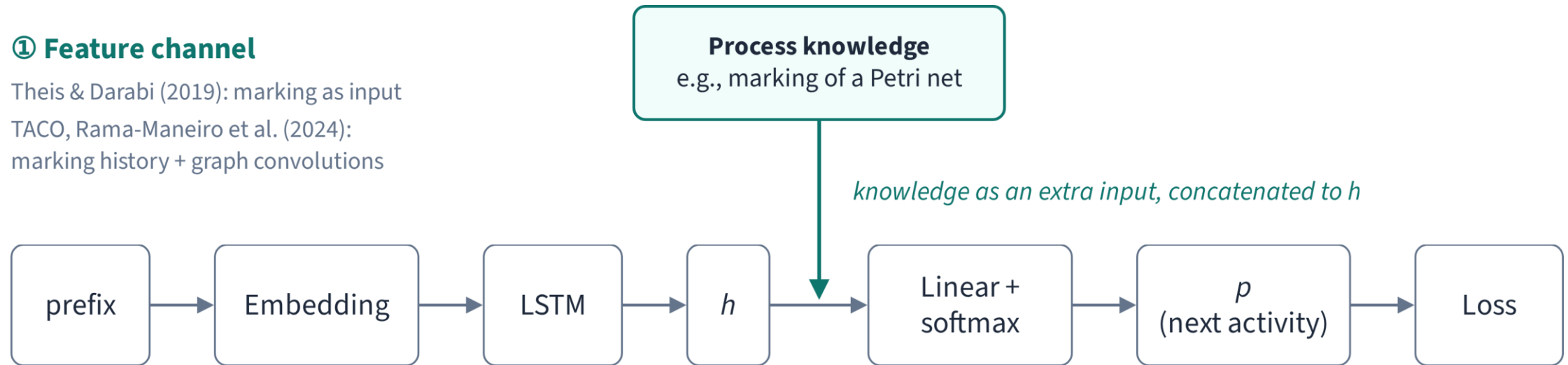
closes G3

Knowledge can enter the network through two channels

① Feature channel

Theis & Darabi (2019): marking as input

TACO, Rama-Maneiro et al. (2024):
marking history + graph convolutions



② Loss channel

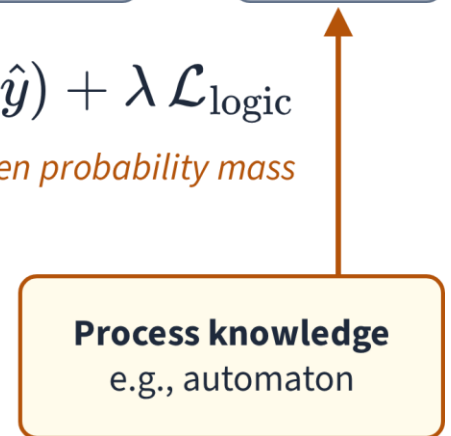
Differentiable temporal logic, Xu et al. (2022) · T-LEAF, Xie et al. (2021)

Mezini et al. (2026): logic losses for PPM **STATE OF THE ART**

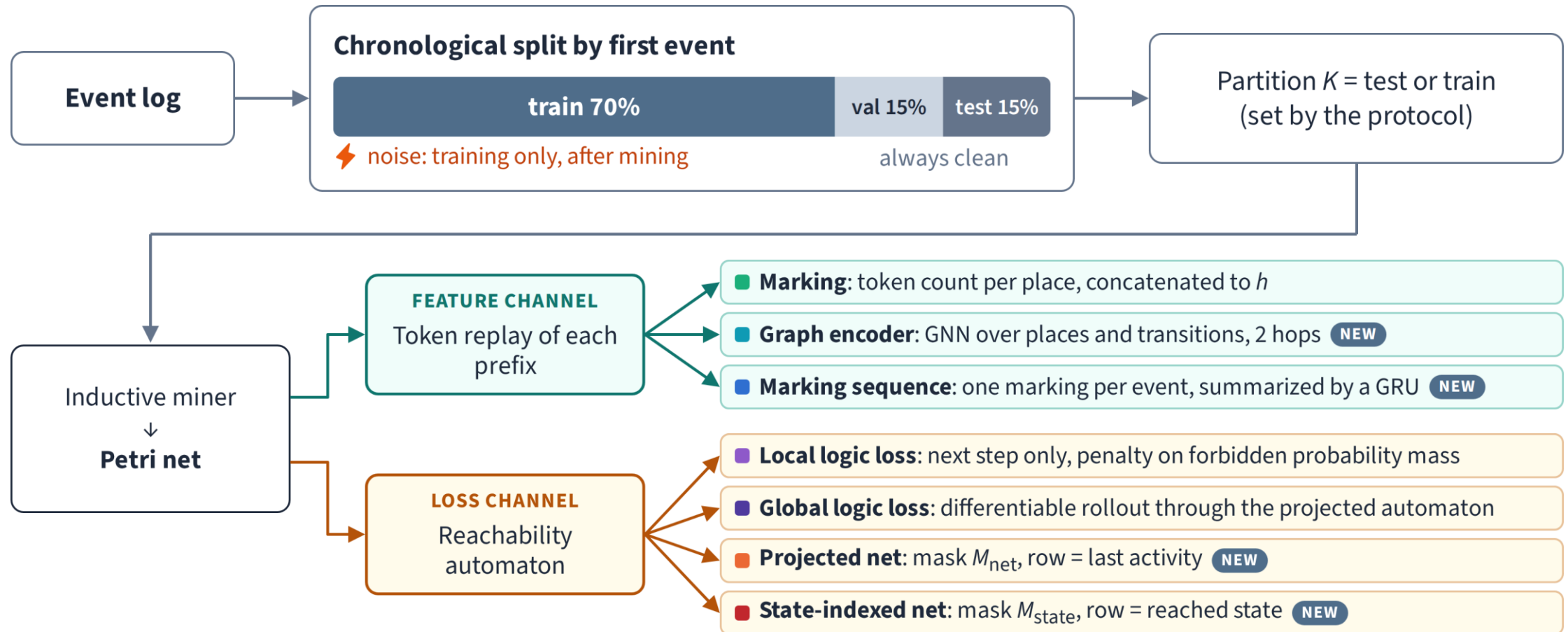
Same loss used so far only with declarative knowledge, never procedural

$$\mathcal{L} = \text{CE}(y, \hat{y}) + \lambda \mathcal{L}_{\text{logic}}$$

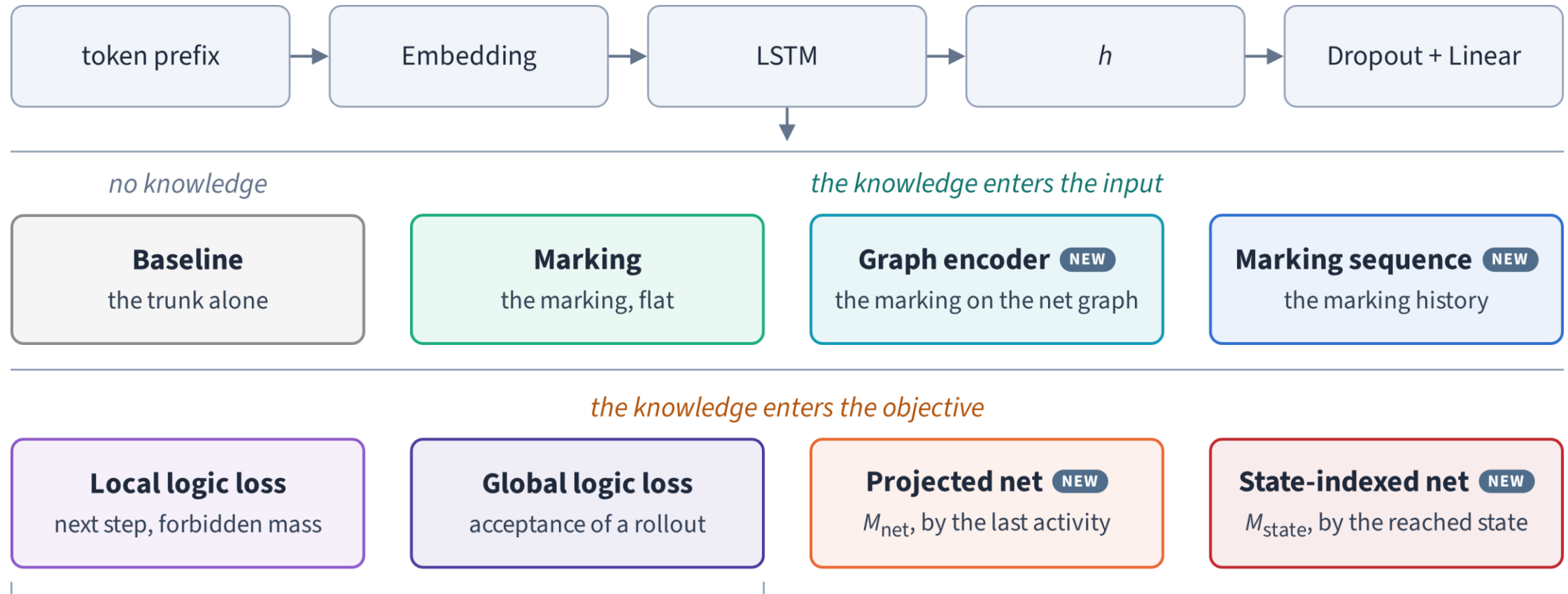
penalty on forbidden probability mass



A single discovered net is the only source of knowledge

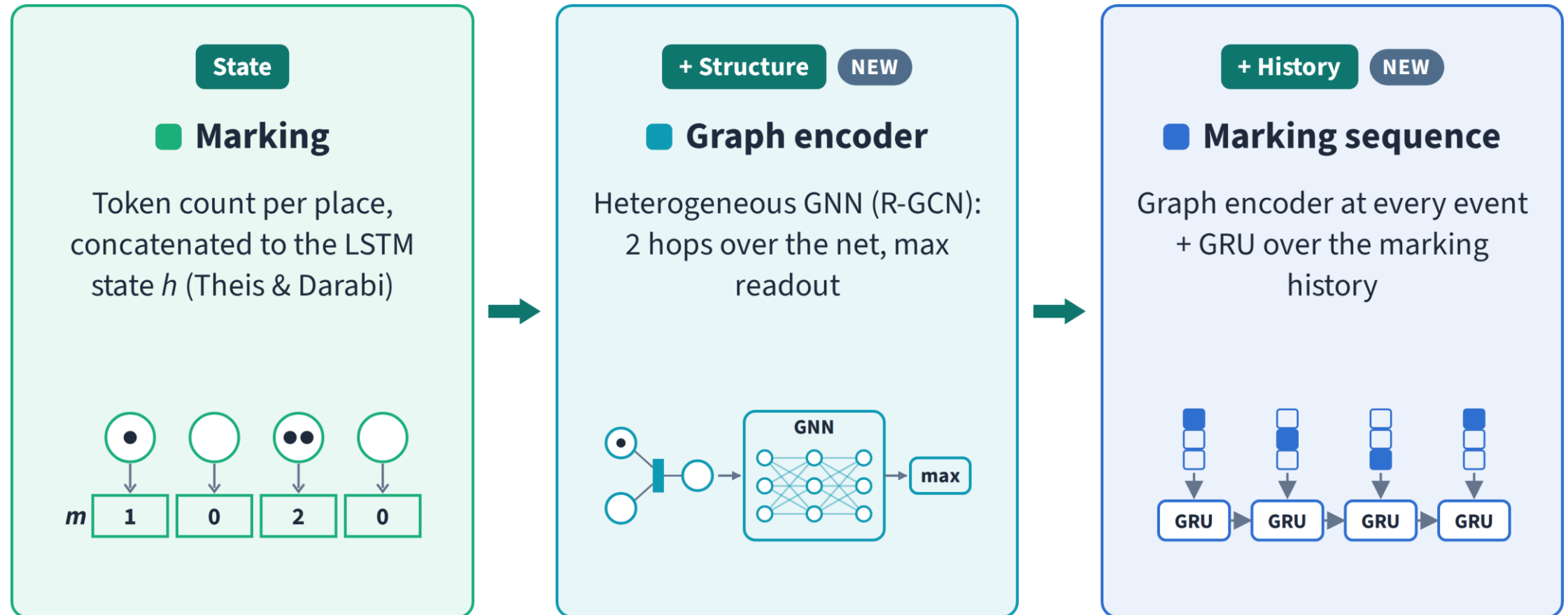


Eight models, one trunk: only the channel changes



from Mezini et al.

Feature channel: three encodings of increasing richness



Objective = cross-entropy only.

Loss channel: four penalties on forbidden behaviour

STATE OF THE ART

Local logic loss

Next step only: penalty on the forbidden probability mass of the prediction

Close forbidden

Approve

Pay

Register forbidden

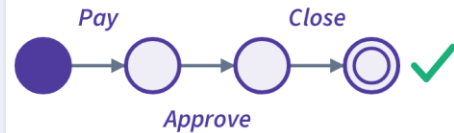
one step: the next activity

STATE OF THE ART

Global logic loss

Rollout of H steps through the projected automaton, made differentiable

sampled rollout (Gumbel-Softmax)



is the whole suffix accepted?

NEW

Projected net

Penalty on the probability that the mask row of the last activity forbids

last	Reg	Pay	Chk	App	Cls
Register	0	1	1	0	0
Pay	0	0	1	1	1
Check	0	1	0	1	0
Approve	0	1	0	0	1

row = last activity

NEW

State-indexed net

Same penalty, with one mask row per state of the automaton

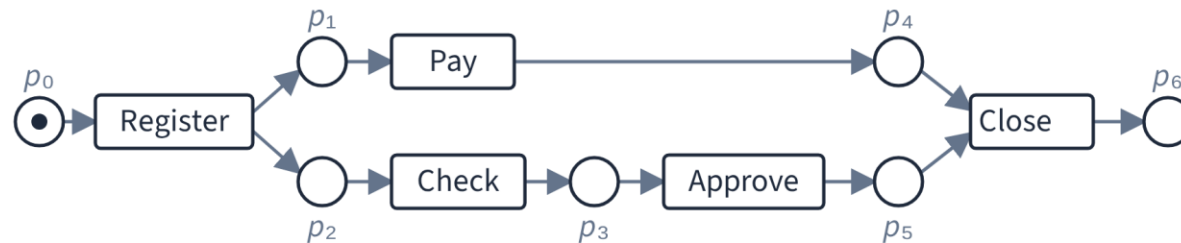
state	Reg	Pay	Chk	App	Cls
q1	0	1	1	0	0
q2	0	1	0	1	0
q3	0	1	0	0	0
q4	0	0	0	0	1

row = reached state

All four keep the baseline architecture: only the objective differs.

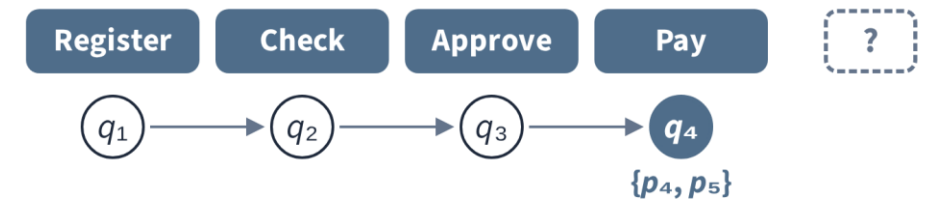
Logic loss based channel: from Petri net to automaton

One net



Pay and the Check → Approve branch interleave in any order.

One prefix, replayed



Reachability automaton: states are markings of the net, edges are activities.

Two masks, read from the same automaton

Projected mask M_{net} · row = last activity = Pay

Pay can fire at three points of the case, so the row allows the union of what the three states it reaches allow.

- Pay before Check → allows **Check**
- Pay between Check and Approve → allows **Approve**
- Pay after Approve → allows **Close**

Row allows:



information lost: the row cannot tell the three apart

State-indexed mask M_{state} · row = state reached = $q_4 = \{p_4, p_5\}$

The replay of the prefix knows that Check and Approve are done.

Row allows: **Close** ~~Check~~ ~~Approve~~

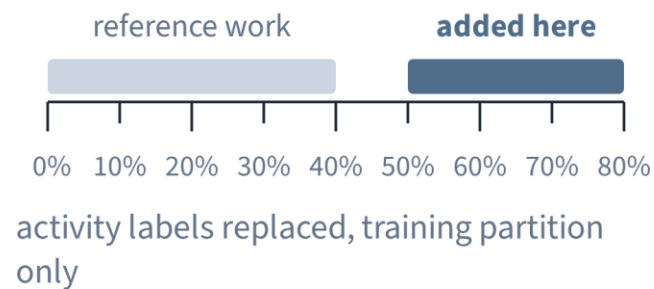
information kept: the state tells the three apart

RQ2 asks whether that extra information is worth keeping.

5,760 runs: 8 models $\times$ 4 logs $\times$ 9 noise levels $\times$ 10 seeds $\times$ 2 protocols

Log	Domain	Cases	Events	Activities	Median length
Sepsis Cases	Hospital	1,050	15,214	16	13
BPIC 2013 incidents	IT service desk	7,554	65,533	4	6
BPIC 2020 domestic declarations	Travel expenses	10,500	56,437	17	5
BPIC 2012	Loan applications	13,087	262,200	24	11

Noise



Protocols

Test protocol: knowledge mined from the test partition (as in the reference work)

Train protocol: knowledge mined from the training partition (as in deployment)

Metrics

Prediction: next-activity accuracy $\cdot$ suffix Damerau-Levenshtein similarity

Conformance: forbidden probability mass $\cdot$ suffix violation rate

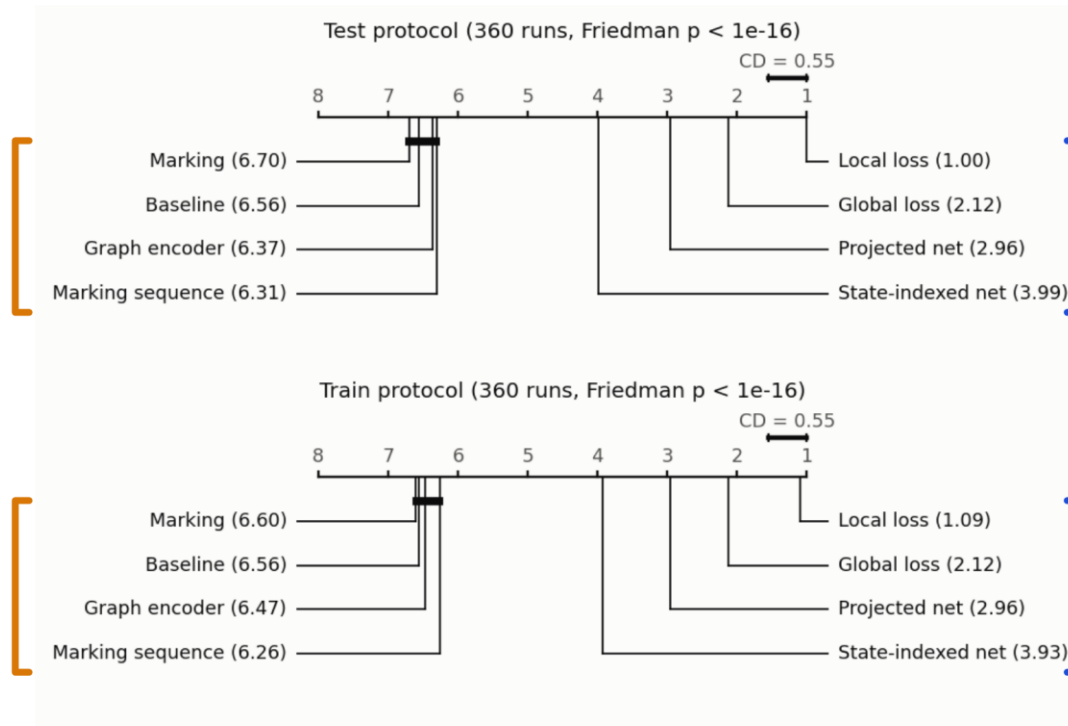
Cost: seconds per epoch

Only the loss channel reduces violation rates

	Test protocol				Train protocol			
	Prediction (↑ better)		Violation Rates (↓ better)		Prediction (↑ better)		Violation Rates (↓ better)	
	Accuracy	DL similarity	Forbidden mass	Suffix violations	Accuracy	DL similarity	Forbidden mass	Suffix violations
LOSS CHANNEL								
Local loss	-1.3% 13/36	-3.9% 9/36	-99.0% 36/36	-95.1% 36/36	+0.8% 12/36	-1.5% 12/36	-98.9% 36/36	-92.2% 36/36
Global loss	+1.6% 24/36	+1.3% 21/36	-32.2% 36/36	-88.2% 35/36	+1.5% 23/36	+1.1% 18/36	-34.0% 36/36	-98.7% 36/36
Projected net	+1.2% 22/36	+0.5% 25/36	-23.9% 36/36	-19.3% 28/36	+1.2% 25/36	+0.7% 23/36	-23.8% 36/36	-18.0% 29/36
State-indexed net	+1.2% 27/36	+1.7% 25/36	-11.2% 36/36	-7.3% 25/36	+1.2% 25/36	+1.7% 26/36	-15.0% 36/36	-3.6% 29/36
FEATURE CHANNEL								
Marking	+0.1% 18/36	-0.1% 18/36	+0.6% 15/36	+14.0% 19/36	+0.0% 21/36	+0.1% 15/36	+0.7% 14/36	+12.0% 23/36
Graph encoder	+0.1% 19/36	+0.1% 24/36	-0.1% 17/36	+6.7% 17/36	+0.1% 22/36	+0.1% 26/36	-0.1% 20/36	+3.1% 21/36
Marking sequence	-0.3% 14/36	+0.0% 18/36	-0.3% 23/36	-5.7% 23/36	-0.1% 16/36	+0.1% 19/36	-0.6% 26/36	-6.8% 26/36

blue = better, orange = worse than baseline · n/36 = settings in which the model is better than or equal to the baseline
Mean difference vs baseline over all logs and noise levels; each setting = median of 10 seeds.

RQ1: Every loss separates from the baseline, no feature model does



one group: the test cannot separate them | each one separated from every other model

Every run ranks the eight models by forbidden mass, 1 is best;
The axis is the average rank over the 360 runs of a protocol.

4 of 4

Losses separated from the baseline and from every feature model; the smallest gap, 2.3 ranks, is four times the critical difference

1.00 / 1.09

Average rank of the local loss, test and train protocol: first in nearly every one of the 360 runs

0.39 ranks

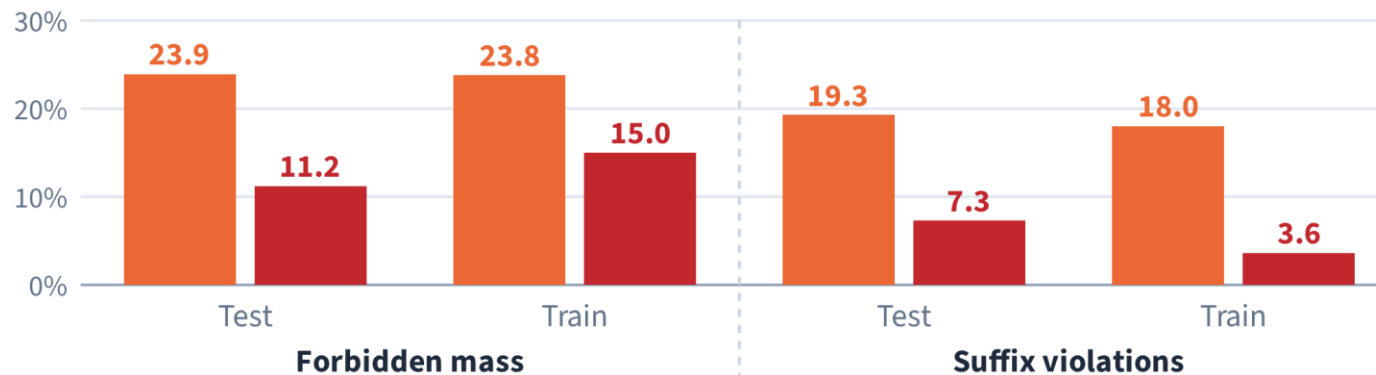
Spread of the baseline and the three feature models, under one critical difference; the four losses spread over three ranks

RQ2: Keeping the automaton's information does not pay

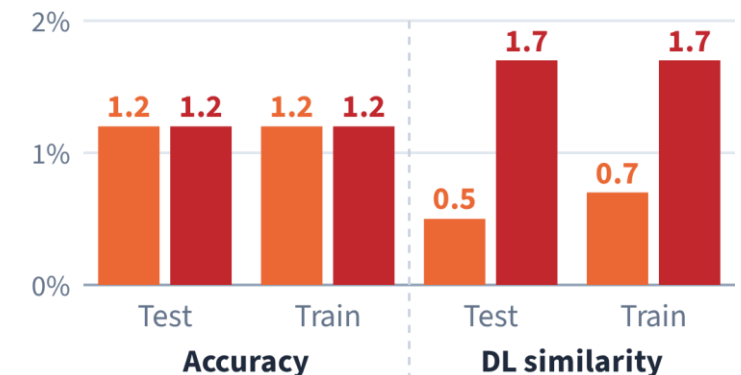
Same trunk, same net, same penalty and weight: only the indexing of the mask changes.

Projected net State-indexed net

Violation Rates: % of the baseline removed (higher is better)



Prediction: % vs baseline

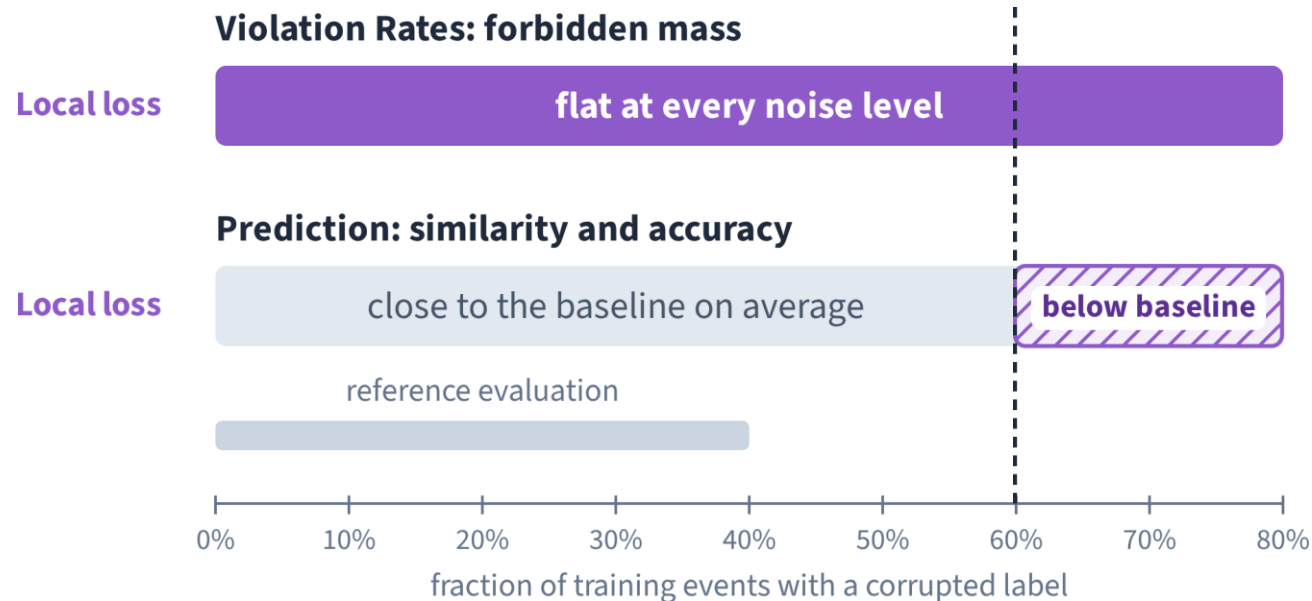


All four losses, forbidden mass removed (test protocol; the train protocol gives the same order)



The local loss and the projected net read the same mask: what sets the size of the drop is the form of the penalty, not how much of the net it reads.

RQ2: Beyond 60% noise, the strongest penalty starts to pay



≤ 0.010

local loss forbidden mass, at every noise level

96%

settings where its suffix violation rate is exactly zero

$> 60\%$

noise level beyond which it loses similarity and accuracy against the baseline

RQ3: The most effective variant trains at the baseline's price

Seconds of one full training run · mean of 720 runs per model: 4 logs × 9 noise levels × 10 seeds × 2 protocols

	Run (s)	Epoch (s)	Epochs	× baseline
Baseline	101	7.4	12.5	1.00
LOSS CHANNEL				
Local loss	84	7.5	11.6	1.00 ± 0.46
Global loss	246	16.1	14.5	2.68 ± 1.06
Projected net	102	7.3	12.6	1.03 ± 0.34
State-indexed net	108	7.4	13.2	1.09 ± 0.38
FEATURE CHANNEL				
Marking	111	7.6	12.9	1.17 ± 0.46
Graph encoder	111	7.9	12.8	1.27 ± 0.53
Marking sequence	306	21.3	12.2	2.53 ± 1.16

blue = faster, orange = slower than the baseline · × baseline is measured run by run inside the same cell, so the size of the log cancels out. Computational time measured on one NVIDIA A100.

What that time buys

Local loss 1.00× the baseline

removes > 90% of the forbidden mass and of the suffix violations

Global loss 2.7×

removes > 93% of the violations and a third of the mass

Marking sequence 2.5×

removes nothing

Where the knowledge enters decides whether it has any effect

RQ1 · Channel

The same Petri net as a feature: no effect on conformance, at up to 2.2× the time per epoch.

RQ2 · Form

The simple projection beats the state-indexed automaton; penalties rank by how hard they push, not by expressive power.

RQ3 · Cost

The cheapest variant (local loss) is the most effective but pays beyond 60% noise; the global loss improves conformance with no loss in prediction, at 2.3× the time per epoch.

Same answers under both protocols.

Limitations and Future work

Limitations

- One automaton scores all models, and local loss and projected net train on that same mask: part of their gain is by construction
- A mined net says what the log did, not what the process should do: no expert knowledge tested
- Generation stops at the true length: similarity compares our eight models

Future work

- Declarative vs procedural: a Declare model compiled into an automaton
- Decode-time masking as the missing baseline: zero violations, no retraining. What does training add beyond it?
- An end-of-trace token: comparable suffix results, and does knowledge help decide when a case ends?

Thank you

Questions are welcome!

github.com/MatteoZorzi/neuro-symbolic-ppm-thesis
code, scripts and result grids

Matteo Zorzi • unibz



[Link to GitHub](https://github.com/MatteoZorzi/neuro-symbolic-ppm-thesis)